



Agent SDK Hooks

การวัดพฤติกรรมแบบ Deterministic ใน LLMs กับ Agent

Domain 1 · Task 1.5 — Claude Certified Architect

Agent SDK Hooks คืออะไร?

- คือกลไกสำหรับ **จัดการพฤติกรรมแบบ deterministic** เข้าไปในระบบที่โดยธรรมชาติแล้วเป็น **probabilistic**
- ทำงานอยู่ตรง **boundary** ระหว่างการตัดสินใจของโมเดล กับการนำไปปฏิบัติจริงในโลกภายนอก

หน้าที่หลัก

เข้าไป **ดักจับ (intercept)** การเรียก tool และผลลัพธ์ของ tool เพื่อ:

บังคับใช้ business rule

ทำให้ข้อมูลเป็นมาตรฐานเดียวกัน

สองประเภทของ Hook

⏮ PreToolUse

ทำงาน **ก่อน** tool ถูกเรียกใช้ (ดักจับ tool-call)

block การเรียก

modify การเรียก

redirect เวิร์กโฟลว์

ถ้า hook block ไว้ → tool จะ**ไม่มีวันทำงาน**

⏭ PostToolUse

ทำงาน **หลัง** tool เสร็จ แต่ **ก่อน** โมเดลจะเห็นผลลัพธ์

ใช้ดักและ **transform** ผลลัพธ์ของ tool ให้โมเดลได้รับข้อมูลที่สะอาด และเป็นมาตรฐานเดียวกันเสมอ — ไม่ว่ามาจาก tool ตัวไหน



Key Concept: Deterministic vs Probabilistic

หัวใจของ Task นี้: เลือกกลไกให้ตรงกับ**ระดับการรับประกัน**

✓ ต้องเป็นตามนั้น 100% ทุกครั้ง

→ ใช้ **hooks** (deterministic)

~ อยากรให้เป็น แต่หลุดบ้างรับได้

→ ใช้ **prompts** (probabilistic)

📌 จำประโยคสอบให้แม่น

"PostToolUse hooks **transform data** after execution.

PreToolUse hooks **enforce policy** before execution."

PostToolUse: Data Normalisation

ปัญหา

MCP tool แต่ละตัวคืนข้อมูลคนละฟอร์แมต

1710489600 VS "2024-03-15T12:00:00Z" | 200 / 404 / 500 VS "active" / "pending"

✅ วิธีแก้ — PostToolUse hook

แปลงทุกฟอร์แมตให้เป็นมาตรฐานเดียวก่อนถึงมือโมเดล

- Unix timestamp → ISO 8601
- status code ตัวเลข → สตริงที่อ่านออก
- ค่าเงิน → ทศนิยม + currency code รูปแบบเดียว

ผล: โมเดลไม่ต้องเดาฟอร์แมตเอง → ไม่เกิด **interpretation error**

PreToolUse: Policy Enforcement

3 use cases — block ก่อน tool ทำงานจริง

1. Refund Threshold

ดัก `process_refund` ถ้าเกิน **\$500** → block ทันที แล้ว redirect ไป human escalation — tool ไม่ทำงานเลย

2. Compliance Prerequisite Gate

ดัก `transfer_funds` ถ้า **AML check** ของ session ยังไม่เสร็จ → block แล้วคืน error สั่งให้ทำ AML ให้เสร็จก่อน

3. Manager Approval Workflow

ดัก `approve_discount` ที่เกิน **20%** → หยุดไว้ แล้ว route เข้า queue ให้ผู้จัดการอนุมัติ — tool ทำงานต่อเมื่ออนุมัติแล้ว

Hooks vs Prompts: กรอบการตัดสินใจ

หลักวินิจัย

อย่าถามว่า "prompt ดีพอไหม" — ให้ถามว่า

"ความเสียหายจากการหลุดแค่ครั้งเดียว คำนึงกับการรับประกันแบบ deterministic หรือไม่?"

💸 เสียเงินจากความล้มเหลวครั้งเดียว

→ hooks

⚖️ เสี่ยงทางกฎหมายจากครั้งเดียว

→ hooks

🎨 แค่ความชอบเรื่องรูปแบบ/สไตล์

→ prompt พอ

ตัวอย่าง: **AML ก่อน international transfer** และ **refund เกิน \$500** ต้องใช้ hook (95% ไม่พอ) · แต่การจัด **markdown format** ใช้ prompt พอ —
ใช้ hook = overhead เกินจำเป็น



Exam Trap หลัก: PostToolUse ไม่ block



ออกสอบบ่อย

ตัวหลอก

ข้อสอบชอบเสนอ `PostToolUse` hook เป็นทางแก้สำหรับ "block การกระทำที่ผิดนโยบาย"

× **ผิด!** เพราะ `PostToolUse` ทำงาน **หลัง** tool รันไปแล้ว — กว่ามันจะ fire การกระทำที่ไม่ compliant ก็เกิดขึ้นเรียบร้อยแล้ว



หลักวิธีจ้ย

อะไรที่ต้อง "กันไม่ให้เกิด"

→ ใช้ `PreToolUse` เสมอ

`PostToolUse` ได้แค่ "แปลงผล / ตั้ง flag ที่หลัง"

→ สำหรับการละเมิดที่เกิดขึ้นแล้ว **สายเกินแก้**



Case Study: Data Format Chaos

อาการ

3 MCP tool ค้นข้อมูลคนละแบบ:

- `get_customer` — Unix timestamp + status ตัวเลข
- `lookup_order` — ISO 8601 + สตริงอังกฤษ
- `check_shipping` — DD/MM/YYYY + โคลดตัวเดียว ("S"=shipped, "P"=pending)

สาเหตุ

ไม่มี hook → โมเดลต้องตีความ 3 ฟอแมตวันที่ + 3 แบบ status ทุกรอบ → **มั่ว**(สลับวัน/เดือน, อ่าน "P" เป็น "processed" แทน "pending")

✅ วิธีแก้

PostToolUse hook **normalize** ทุกผลก่อนถึงโมเดล → วันที่เป็น ISO 8601, status เป็นสตริงที่อ่านออก → interpretation error หายทั้งหมด

! Exam Traps ที่ต้องระวัง

1. ใช้ PostToolUse เพื่อ block การกระทำที่ผิดนโยบาย

× ผิด — มัน fire หลังการกระทำเกิดไปแล้ว ต้องใช้ PreToolUse

2. เพิ่มคำสั่งใน prompt ให้ละเอียดขึ้นเพื่อหวัง compliance 100%

× prompt ให้ได้แค่ probabilistic — งานการเงิน/กฎหมาย/security ต้องใช้ hook

3. ให้โมเดลแปลงข้อมูลเอง (model-side transformation) แทน PostToolUse

× การให้โมเดล normalize เองทำให้ข้อมูลไม่สม่ำเสมอ

4. สับสนทิตทางของ hook

$\text{PostToolUse} = \text{แปลงหลังรับ} \cdot \text{PreToolUse} = \text{block/แก้ก่อนรับ}$ — เลือกผิดทิศ =พลาดการป้องกัน



Practice Scenario

โจทย์

agent บางครั้งประมวลผล **international transfer** โดยไม่ได้ทำ compliance check ที่กำกับต้องการบังคับใช้ **AML check ให้ครบ 100%** ก่อนทุก transfer ระบบปัจจุบันใช้คำสั่งใน prompt ซึ่งได้ผลราว **95%** — ควรทำอย่างไร?

- A. เพิ่มคำสั่ง AML ใน system prompt ให้ละเอียด พร้อมตัวอย่างและคำเตือนเรื่องบทลงโทษ
- B. ทำ PreToolUse hook ที่ block `transfer_funds` จนกว่า `aml_check` จะคืนค่า pass
- C. เพิ่ม PostToolUse hook คอยตั้ง flag transfer ที่ข้าม AML ไว้ให้คนมารีวิวทีหลัง
- D. ระบุ agent ด้วย few-shot examples แสดงเวิร์กโฟลว์ AML ที่ถูกต้องก่อนทุก transfer



หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option B

PreToolUse hook ที่ block `transfer_funds` จนกว่า `aml_check` จะ pass ให้การบังคับใช้แบบ **deterministic 100%** ตามที่ทีมกำกับต้องการ — ไม่มี transfer ใดผ่านไปได้โดยไม่ผ่าน AML

A & D — ยังเป็น probabilistic ทั้งคู่ (prompt / few-shot ไม่การันตี 100% — 95% ก็ยังหลุด)

C — PostToolUse fire **หลัง** transfer เกิดไปแล้ว การละเมิดเกิดก่อน flag ถูกตั้งเสมอ จึงสายเกินไป



Build Exercise

โจทย์ 60 นาที — สรุปเชิงปฏิบัติ

1 สร้าง 3 MCP tool ค้นข้อมูลและฟอร์แมต (Unix / ISO 8601 / DD/MM/YYYY)

2 เขียน PostToolUse hook normalize ทุกผลเป็น ISO 8601 + สตริงอังกฤษ

3 ยืนยันว่าโมเดลได้ข้อมูลสม่ำเสมอ

4 เพิ่ม PreToolUse hook block `process_refund` เมื่อเกิน \$500

5 เพิ่ม PreToolUse hook block `transfer_funds` จนกว่า `aml_check` จะ pass

6 ทดสอบว่า tool ที่ถูก block "ไม่ทำงานจริง" (ไม่ใช่แค่ log) และทำงานได้เมื่อ prerequisite ครบ



บทเรียนหลัก: PreToolUse บังคับนโยบาย**ก่อนรัน** · PostToolUse แปลงข้อมูล**หลังรัน** — และ hook ให้การรับประกันที่ prompt ให้ไม่ได้



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

agent บางครั้งประมวลผล international transfer โดยข้าม compliance check ที่กำกับต้องการบังคับใช้ anti-money laundering (AML) check ให้ครบ **100%** ก่อนทุก international transfer ระบบปัจจุบันใช้คำสั่งใน prompt ซึ่งได้ผลราว **95%** — วิธีที่ถูกต้องคืออะไร?

A. เพิ่มคำสั่ง AML check ใน system prompt ให้ละเอียด พร้อมตัวอย่างพฤติกรรมที่ถูกและคำเตือนเรื่องบทลงโทษ

B. ทำ PreToolUse hook ที่ block tool `transfer_funds` จนกว่า `aml_check` จะคืนผล verified pass

C. เพิ่ม PostToolUse hook ที่ตั้ง flag transfer ที่ข้าม AML check ไว้ให้คนมารีวิวทีหลัง

D. Instruct agent ด้วย few-shot examples แสดงเวิร์กโฟลว์ AML verification ที่ถูกต้องก่อนทุก transfer

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option B

PreToolUse hook ดักจับ tool call ก่อนถูก execute และ block ไว้จนกว่า AML check จะผ่าน จึงให้การรับประกันแบบ deterministic 100% ตามที่ regulation ต้องการ — ไม่มี transfer ใดผ่านไปได้โดยไม่ผ่าน verification

ทำไมข้ออื่นผิด

A — prompt ที่ละเอียดขึ้นอาจดันจาก 95% เป็น 97-98% แต่ไปไม่ถึง 100% กับ AML แค่พลาดครั้งเดียวก็เสี่ยงบดทอนทางกฎหมายหนัก probabilistic ไม่พอ

C — PostToolUse รับหลัง execution กว่า hook จะจับได้ว่า AML หาย transfer ที่ไม่ compliant ก็ถูกประมวลผลไปแล้ว compliance ต้องป้องกัน ไม่ใช่ตรวจย้อนหลัง

D — few-shot examples เพิ่มความแม่นยำแต่ยังเป็น probabilistic การันตี 100% ไม่ได้ ในขณะที่ regulation ต้องการการบังคับใช้แบบ deterministic



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

customer support agent ใช้ MCP tool 3 ตัวที่คืนวันที่คนละฟอร์แมต: Unix timestamp (1710489600), ISO 8601 string ("2024-03-15T12:00:00Z") และ DD/MM/YYYY ("15/03/2024") บางครั้งโมเดลสับสนลำดับ day/month — วิธีแก้ที่ดีที่สุดคืออะไร?

- A. เพิ่มคำสั่งใน system prompt อธิบายฟอร์แมตวันที่ทั้ง 3 แบบและวิธีตีความแต่ละแบบ
- B. ทำ PostToolUse hook ที่ normalise ทุกฟอร์แมตวันที่เป็น ISO 8601 ก่อนโมเดลจะประมวลผล
- C. ทำ PreToolUse hook ที่แปลงวันที่ก่อนส่งไปให้ tool
- D. ใช้ MCP tool เพียงตัวเดียวเพื่อเลี่ยงความไม่สอดคล้องของฟอร์แมต

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option B

PostToolUse hook ดึงผลลัพธ์ของ tool หลัง execution แต่ก่อนโมเดลจะเห็น การ normalise ทุกวันที่เป็น ISO 8601 ตรงจุดนี้ทำให้โมเดลได้ข้อมูลที่สอดคล้องกันเสมอ ไม่ว่ามาจาก tool ตัวไหน

ทำไมข้ออื่นผิด

- A — การฟังให้โมเดลตีความ 3 ฟอर्मแมตถูกต้องทุก iteration ทำให้เกิดความไม่สม่ำเสมอ อาจ parse ถูกครั้งหนึ่งแล้วผิดครั้งถัดไป
- C — PreToolUse รับก่อน tool execution แต่วันที่อยู่ใน tool result (หลัง execution) ไม่ใช่ใน tool call (ก่อน execution) ทิศทางของ hook จึงผิด
- D — การจำกัดเหลือ tool เดียวลดความสามารถของระบบ วิธีที่ถูกคือ normalise output ที่ต่างกัน ไม่ใช่เลี่ยงการใช้หลาย tool



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

ข้อความใดอธิบายความต่างระหว่าง PostToolUse hooks กับ PreToolUse hooks ได้ถูกต้อง?

- A. PostToolUse hooks รับก่อน tool execution ส่วน PreToolUse hooks รับหลัง
- B. PostToolUse hooks transform ผลลัพธ์หลัง execution ส่วน PreToolUse hooks block หรือ modify การเรียกก่อน execution
- C. ทั้งสองแบบรับที่จุดเดียวกัน แต่ PostToolUse จัดการ data ส่วน PreToolUse จัดการ error
- D. PostToolUse hooks ใช้ได้กับ MCP tool เท่านั้น ส่วน PreToolUse ใช้ได้กับ tool ทุกประเภท



หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

PostToolUse hooks รับหลัง tool execute แต่ก่อนโมเดลประมวลผลผลลัพธ์ (เหมาะกับ data normalisation) ส่วน PreToolUse hooks รับก่อน tool execute (เหมาะกับ policy enforcement และการ block action)

ทำไมข้ออื่นผิด

- A — สลับทิศทางกัน PostToolUse คือหลัง execution ไม่ใช่ก่อน และ PreToolUse คือก่อน execution ไม่ใช่หลัง
- C — hook ทั้งสองรับคนละจุดใน lifecycle ไม่ใช่จุดเดียวกัน timing คือความต่างพื้นฐานของมัน
- D — hook ทั้งสองแบบใช้ได้กับ tool ทุกประเภท ความต่างคือ timing (ก่อน vs หลัง execution) ไม่ใช่ความเข้ากันได้กับ tool



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

developer ทำ PostToolUse hook เพื่อ block refund ที่เกิน \$500 ทำไมวิธีนี้ถึงมีข้อบกพร่อง?

- A. PostToolUse hooks เข้าถึง parameter จำนวนเงิน refund ไม่ได้
- B. PostToolUse hooks รันหลัง tool execute ไปแล้ว — refund ถูกประมวลผลไปเรียบร้อยแล้วตอนที่ hook fire
- C. PostToolUse hooks มีให้ใช้เฉพาะใน paid tier ของ Agent SDK เท่านั้น
- D. PostToolUse hooks ทำได้แค่ transform data ไม่สามารถ block operation

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

PostToolUse hooks รับหลัง tool execution ตอนที่ hook fire ตัว `process_refund` ก็ execute และ refund ถูกประมวลผลไปแล้ว การจะ block action ต้องใช้ PreToolUse hook ที่ติดก่อน execution

ทำไมข้ออื่นผิด

- A — PostToolUse hooks เข้าถึง tool result เต็มรวมทั้ง parameter ได้ ปัญหาอยู่ที่ timing ไม่ใช่การเข้าถึงข้อมูล
- C — ไม่ใช่ข้อจำกัดเรื่อง pricing tier ปัญหาคือ PostToolUse hooks รับช้าเกินไปจนป้องกัน action ไม่ทัน
- D — แม้ PostToolUse มักใช้ transform แต่ประเด็นหลักคือมันรับหลัง execution ต่อให้ block ได้ action ก็เกิดไปแล้ว



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

เมื่อไรควรใช้ hooks แทน prompt instructions ในการบังคับใช้ business rule?

A. เมื่อ rule ซับซ้อนและต้องใช้หลายขั้นตอนในการตรวจสอบ

B. เมื่อ rule เกี่ยวกับความชอบเรื่องรูปแบบหรือ output style guideline

C. เมื่อการละเมิดเพียงครั้งเดียวก่อให้เกิด financial loss, legal risk หรือ security breach

D. เมื่อ agent เข้าถึง tool ได้มากกว่า 5 ตัว

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option C

กรอบการตัดสินใจที่ผลกระทบ hooks ให้การรับประกันแบบ deterministic (100%) prompts ให้เพียง probabilistic ถ้าความล้มเหลวครั้งเดียวก่อ financial loss, legal risk หรือ security breach มีแต่ hooks ที่ให้ระดับความมั่นใจที่ต้องการได้

ทำไมข้ออื่นผิด

- A — ความซับซ้อนของ rule ไม่ได้กำหนดทั่วโลก rule ย่อย (block refund เกิน \$500) อาจต้องใช้ hook เพราะความเสี่ยงทางการเงิน ขณะที่ rule ซับซ้อน (รูปแบบ formatting) อาจใช้แค่ prompt
- B — ความชอบเรื่อง formatting เป็นเรื่อง low-stakes เหมาะกับ prompt-based guidance การใช้ hook คือ overhead เกินจำเป็น
- D — จำนวน tool ไม่เกี่ยวกับการตัดสินใจ hooks vs prompts ผลของการละเมิดครั้งเดียวคือปัจจัยชี้ขาด